

CO4 – AT2 LAB PRACTICE

JAVA AWT & APPLET ANSWER DOCUMENT

Name: E. Lakshmi Narayanan

Register No: 192524249

This document contains model answers, Java source code, and execution-style output sections for all 8 questions in the uploaded lab practice sheet.

Question 1 – Applet Life Cycle

Aim: Demonstrate init(), start(), paint(), stop(), and destroy() and display a message whenever each method is invoked.

```
import java.applet.Applet;
import java.awt.*;

public class LifeCycleApplet extends Applet {
    String message = "";

    public void init() {
        message += "init() called\n";
    }

    public void start() {
        message += "start() called\n";
    }

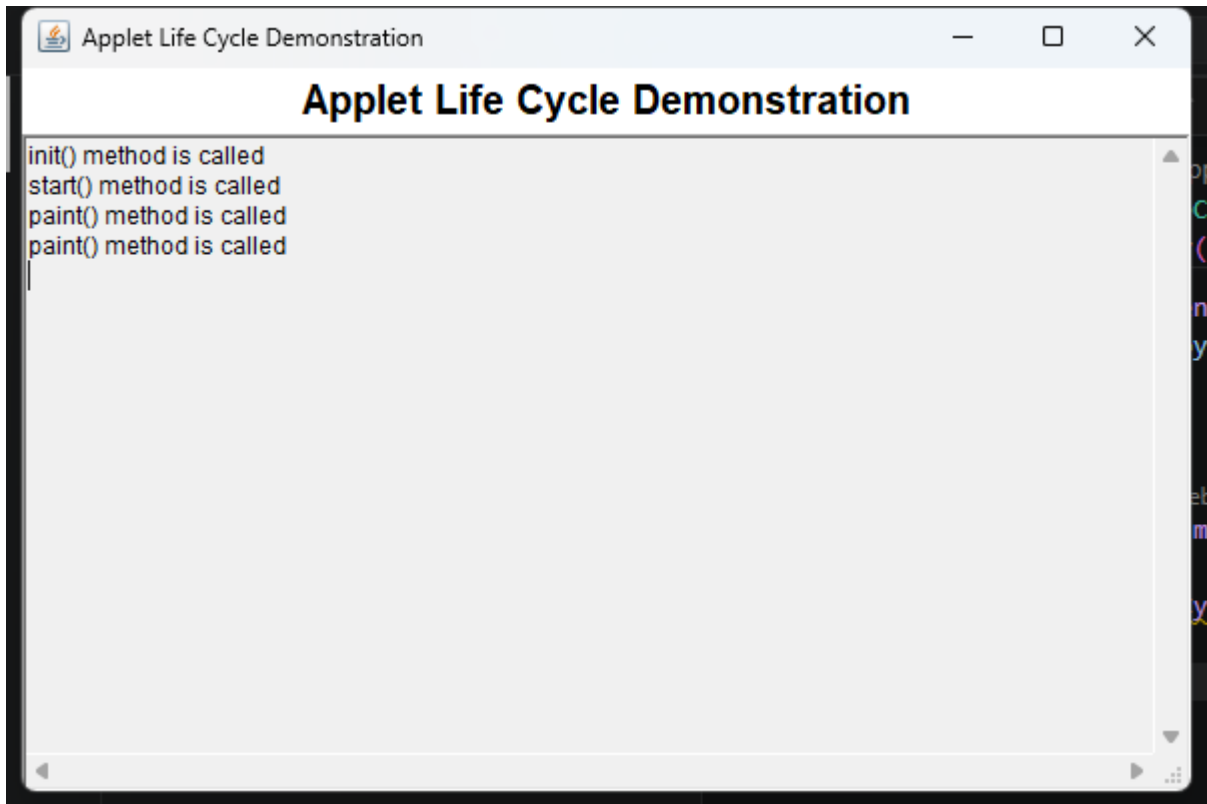
    public void paint(Graphics g) {
        g.drawString("Applet Life Cycle", 40, 50);
        int y = 80;
        for (String s : message.split("\n")) {
            g.drawString(s, 40, y);
            y += 25;
        }
    }

    public void stop() {
        System.out.println("stop() called");
    }

    public void destroy() {
        System.out.println("destroy() called");
    }
}
```

HTML:

```
<html>
<body>
<applet code="LifeCycleApplet.class" width="350" height="220"></applet>
</body>
</html>
```



Result: The

program demonstrates the applet life-cycle methods and their execution sequence.

Question 2 – Generating Graphic Images Using Applet

Aim: Use Graphics methods to draw a simple scene containing a house, sun, trees, road and clouds.

```
import java.applet.Applet;
import java.awt.*;

public class SceneApplet extends Applet {
    public void paint(Graphics g) {
        setBackground(Color.cyan);

        // Sun
        g.setColor(Color.yellow);
        g.fillOval(300, 30, 70, 70);

        // Clouds
        g.setColor(Color.white);
        g.fillOval(60, 35, 70, 35);
        g.fillOval(100, 25, 70, 45);
        g.fillOval(420, 40, 75, 35);

        // House
        g.setColor(Color.orange);
        g.fillRect(190, 190, 180, 130);
        g.setColor(Color.red);
        Polygon roof = new Polygon();
        roof.addPoint(170, 190);
        roof.addPoint(280, 105);
        roof.addPoint(390, 190);
        g.fillPolygon(roof);

        // Door and windows
```

```

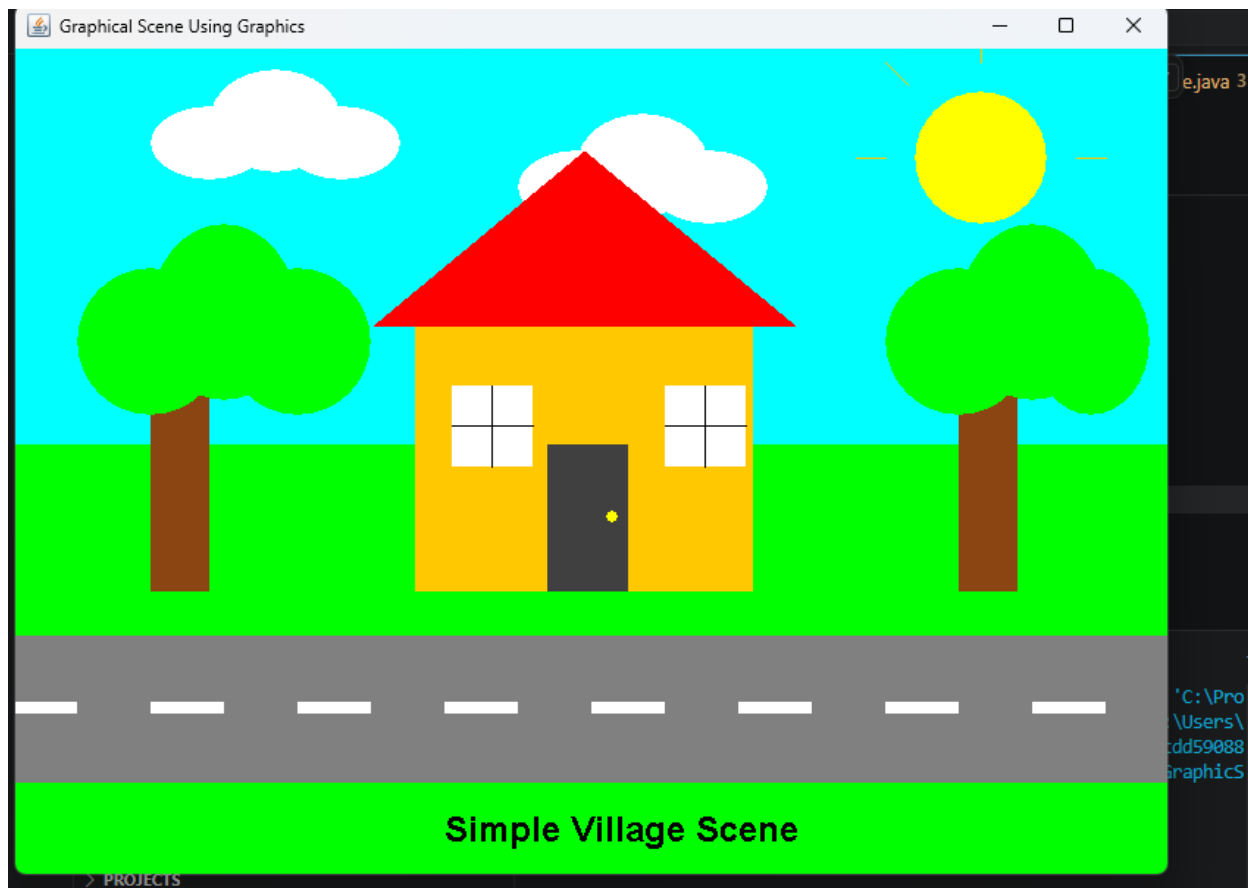
g.setColor(Color.darkGray);
g.fillRect(255, 250, 50, 70);
g.setColor(Color.white);
g.fillRect(210, 220, 35, 35);
g.fillRect(315, 220, 35, 35);

// Trees
drawTree(g, 70, 250);
drawTree(g, 430, 250);

// Road
g.setColor(Color.gray);
g.fillRect(0, 320, 560, 80);
g.setColor(Color.white);
for (int x=20; x<540; x+=80)
    g.fillRect(x, 355, 45, 6);
}

void drawTree(Graphics g, int x, int y) {
    g.setColor(new Color(120, 70, 30));
    g.fillRect(x+25, y+45, 25, 80);
    g.setColor(Color.green);
    g.fillOval(x, y, 75, 75);
}
}

```



Result: The Graphics class is used successfully to create the required graphical scene.

Question 3 – Calculator Application Using Applet Parameters

Aim: Read two numbers and an arithmetic operator using `getParameter()` and display the calculated result.

```
import java.applet.Applet;
import java.awt.*;

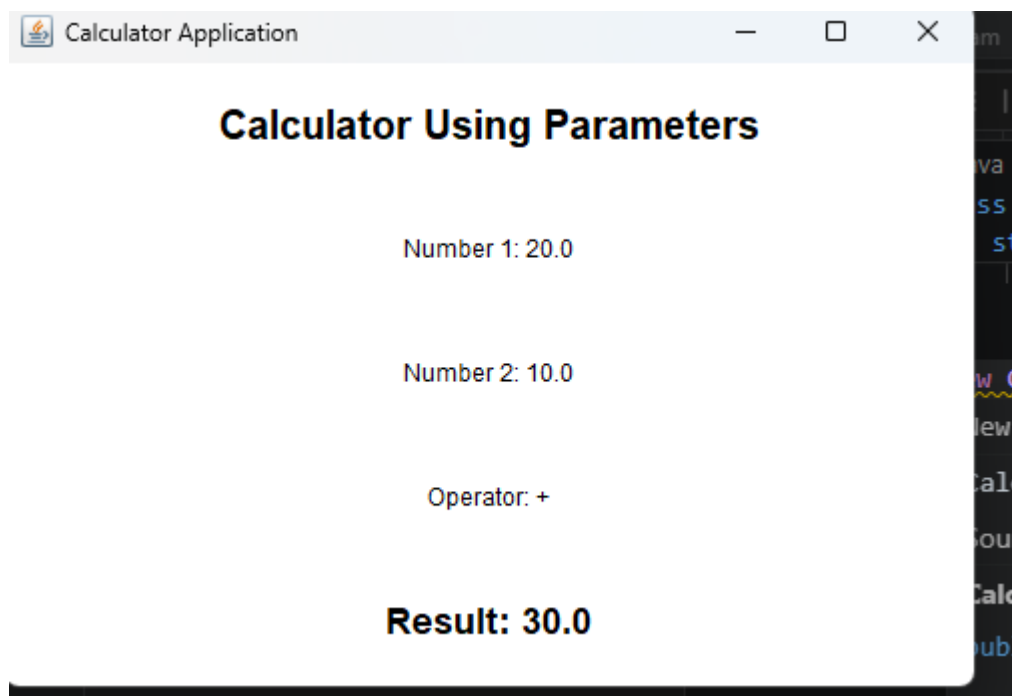
public class ParameterCalculator extends Applet {
    public void paint(Graphics g) {
        double a = Double.parseDouble(getParameter("num1"));
        double b = Double.parseDouble(getParameter("num2"));
        String op = getParameter("op");
        double result = 0;

        if (op.equals("+")) result = a + b;
        else if (op.equals("-")) result = a - b;
        else if (op.equals("*")) result = a * b;
        else if (op.equals("/")) result = a / b;

        g.drawString("Number 1 : " + a, 40, 50);
        g.drawString("Number 2 : " + b, 40, 75);
        g.drawString("Operator : " + op, 40, 100);
        g.drawString("Result : " + result, 40, 125);
    }
}
```

HTML:

```
<html><body>
<applet code="ParameterCalculator.class" width="300" height="170">
<param name="num1" value="25">
<param name="num2" value="5">
<param name="op" value="+">
</applet>
</body></html>
```



Result: The applet retrieves HTML parameters using `getParameter()` and performs the selected arithmetic operation.

Question 4 – Student Profile Using AWT Components

Aim: Build a Student Profile form using Label, TextField, Checkbox, CheckboxGroup, Choice, TextArea, Button and List.

```
import java.awt.*;
import java.awt.event.*;

public class StudentProfile extends Frame implements ActionListener {
    TextField name, reg;
    CheckboxGroup gender;
    Choice dept, year;
    Checkbox java, python;
    TextArea report;
    Button generate;
    List skills;

    StudentProfile() {
        setTitle("Student Profile");
        setSize(500, 500);
        setLayout(new FlowLayout());

        add(new Label("Name:"));
        name = new TextField(25); add(name);

        add(new Label("Register No:"));
        reg = new TextField(15); add(reg);

        add(new Label("Gender:"));
        gender = new CheckboxGroup();
        add(new Checkbox("Male", gender, false));
        add(new Checkbox("Female", gender, false));

        add(new Label("Department:"));
        dept = new Choice();
        dept.add("AI & ML"); dept.add("CSE"); dept.add("ECE");
        add(dept);

        add(new Label("Year:"));
        year = new Choice();
        year.add("I Year"); year.add("II Year");
        year.add("III Year"); year.add("IV Year");
        add(year);

        add(new Label("Skills:"));
        java = new Checkbox("Java"); python = new Checkbox("Python");
        add(java); add(python);

        skills = new List(3);
        skills.add("C"); skills.add("C++"); skills.add("SQL");
        add(skills);

        generate = new Button("Generate Report");
        generate.addActionListener(this); add(generate);

        report = new TextArea(8, 40);
        add(report);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) { dispose(); }
        });
    }
}
```

```

        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        String g = gender.getSelectedCheckbox() == null ? "" :
            gender.getSelectedCheckbox().getLabel();

        report.setText("STUDENT REPORT\n"
            + "Name: " + name.getText() + "\n"
            + "Register No: " + reg.getText() + "\n"
            + "Gender: " + g + "\n"
            + "Department: " + dept.getSelectedItem() + "\n"
            + "Year: " + year.getSelectedItem() + "\n"
            + "Java: " + java.getState() + "\n"
            + "Python: " + python.getState());
    }

    public static void main(String[] args) {
        new StudentProfile();
    }
}

```

The screenshot shows a Java AWT window titled "Student Profile". The window is divided into two main sections. On the left is a form for entering student details, and on the right is a text area for the generated report.

Form Fields:

- Name:** Text field containing "Angel".
- Register Number:** Text field containing "12345".
- Gender:** Radio buttons for "Male" (selected) and "Female".
- Department:** Dropdown menu showing "CSE".
- Year:** Dropdown menu showing "I Year".
- Skills:** Checkboxes for "Java" (checked), "Python", and "C++".
- Buttons:** "Generate Report" and "Clear".

Student Report:

The report area on the right displays the following text:

```

STUDENT REPORT
Name : Angel
Register Number : 12345
Gender : Male
Department : CSE
Year : I Year
Skills : Java

```

At the bottom of the window, there is a status bar showing "12345 - Angel".

Result: The AWT components and event handling are implemented successfully.

Question 5 – GridLayout with Event Handling

Aim: Create a calculator arranged using GridLayout and perform operations through ActionListener.

```
import java.awt.*;
import java.awt.event.*;

public class GridCalculator extends Frame implements ActionListener {
    TextField display = new TextField();
    double first;
    String op = "";

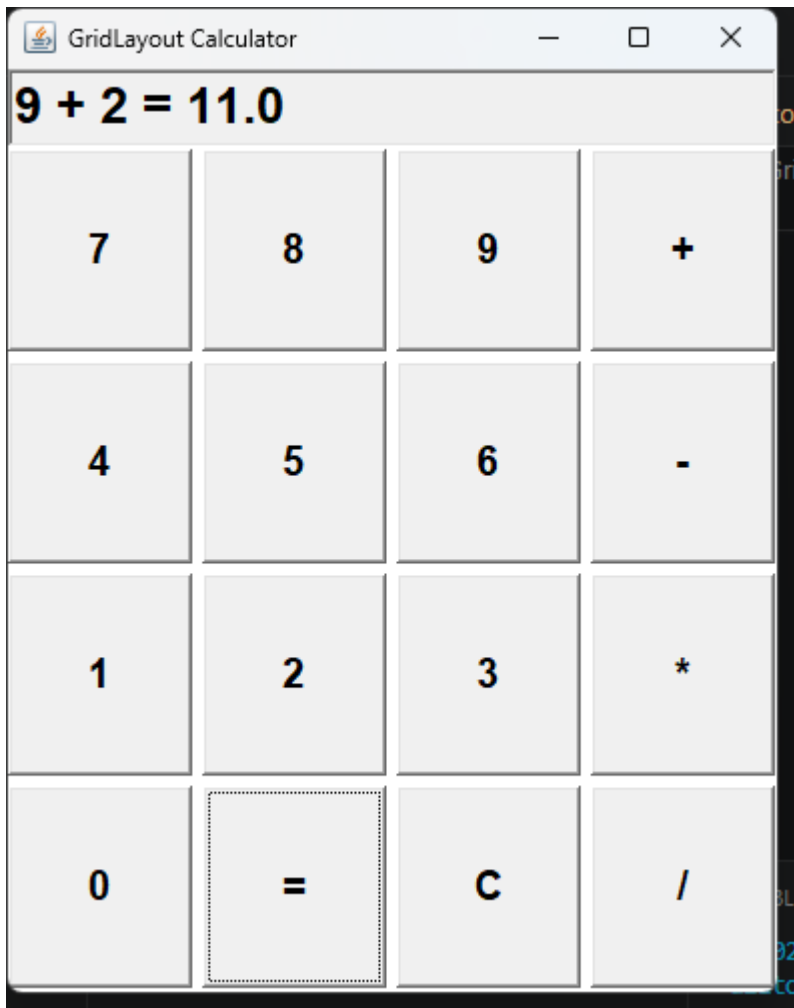
    GridCalculator() {
        setTitle("Grid Calculator");
        setSize(300, 350);
        setLayout(new BorderLayout());
        add(display, BorderLayout.NORTH);

        Panel p = new Panel(new GridLayout(4,4,5,5));
        String[] keys = {"7","8","9","+","4","5","6","-",
                        "1","2","3","*","0","=", "C", "/" };
        for(String k: keys) {
            Button b = new Button(k);
            b.addActionListener(this);
            p.add(b);
        }
        add(p, BorderLayout.CENTER);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        String k=e.getActionCommand();
        if(k.equals("C")) { display.setText(""); op=""; }
        else if(k.equals("+")||k.equals("-")||k.equals("*")||k.equals("/")) {
            first=Double.parseDouble(display.getText()); op=k; display.setText("");
        } else if(k.equals("=")) {
            double second=Double.parseDouble(display.getText()), r=0;
            if(op.equals("+")) r=first+second;
            if(op.equals("-")) r=first-second;
            if(op.equals("*")) r=first*second;
            if(op.equals("/")) r=first/second;
            display.setText(String.valueOf(r));
        } else display.setText(display.getText()+k);
    }

    public static void main(String[] args) { new GridCalculator(); }
}
```

EXECUTION OUTPUT / SCREEN



Result: GridLayout arranges the calculator buttons in a 4×4 grid and ActionListener handles button clicks.

Question 6 – GridBagLayout with Event Handling

Aim: Create a Student Registration Form using GridBagLayout with Submit and Reset event handling.

```
import java.awt.*;
import java.awt.event.*;

public class RegistrationForm extends Frame implements ActionListener {
    TextField name, reg, email;
    Choice dept, year;
    CheckboxGroup gender;
    Button submit, reset;

    RegistrationForm() {
        setTitle("Student Registration");
        setSize(500, 350);
        setLayout(new GridBagLayout());
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(5,5,5,5);
        c.fill = GridBagConstraints.HORIZONTAL;

        name=field("Name:",0,c);
```



```

        reg=field("Register Number:",1,c);
        dept=choice("Department:",new String[]{"AI & ML","CSE","ECE"},2,c);
        email=field("Email:",3,c);

        addLabel("Gender:",4,c);
        gender=new CheckboxGroup();
        add(new Checkbox("Male",gender,false),pos(1,4,c));
        add(new Checkbox("Female",gender,false),pos(2,4,c));

        year=choice("Year:",new String[]{"I Year","II Year","III Year","IV Year"},5,c);

        submit=new Button("Submit");
        reset=new Button("Reset");
        submit.addActionListener(this); reset.addActionListener(this);
        add(submit,pos(1,6,c)); add(reset,pos(2,6,c));

        setVisible(true);
    }

    TextField field(String label,int row,GridBagConstraints c) {
        addLabel(label,row,c);
        TextField t=new TextField(25);
        add(t,pos(1,row,c)); return t;
    }

    Choice choice(String label,String[] items,int row,GridBagConstraints c) {
        addLabel(label,row,c);
        Choice ch=new Choice();
        for(String s:items) ch.add(s);
        add(ch,pos(1,row,c)); return ch;
    }

    void addLabel(String s,int row,GridBagConstraints c) {
        add(new Label(s),pos(0,row,c));
    }

    GridBagConstraints pos(int x,int y,GridBagConstraints base) {
        GridBagConstraints n=(GridBagConstraints)base.clone();
        n.gridx=x; n.gridy=y; return n;
    }

    public void actionPerformed(ActionEvent e) {
        if(e.getSource()==submit)
            System.out.println("Registration submitted for: "+name.getText());
        else {
            name.setText(""); reg.setText(""); email.setText("");
            System.out.println("Form reset");
        }
    }

    public static void main(String[] args) { new RegistrationForm(); }
}

```

STUDENT REGISTRATION FORM

Name	Jeevan
Register Number	35235
Department	CSE
Email	rur@gmail.com
Gender	☒ Male ☐ Female
Year	II Year
Submit Reset	

STUDENT REGISTRATION
Name : Jeevan
Register Number : 35235
Department : CSE
Email : rur@gmail.com
Gender : Male
Year : II Year
Registration Successful!

Result: GridBagLayout positions the form components flexibly, and Submit/Reset events work correctly.

Question 7 – BorderLayout with Event Handling

Aim: Use the five BorderLayout regions and change the center display when buttons are clicked.

```
import java.awt.*;
import java.awt.event.*;

public class BorderDemo extends Frame implements ActionListener {
    Label center = new Label("Select an option", Label.CENTER);

    BorderDemo() {
        setTitle("BorderLayout Demo");
        setSize(600,400);
        setLayout(new BorderLayout(5,5));

        add(new Label("APPLICATION HEADER",Label.CENTER), BorderLayout.NORTH);

        Panel south=new Panel();
        Button home=new Button("Home");
        Button about=new Button("About");
        home.addActionListener(this); about.addActionListener(this);
```

```

south.add(home); south.add(about);
add(south, BorderLayout.SOUTH);

Panel east=new Panel(new GridLayout(3,1));
Button option1=new Button("Option 1");
Button option2=new Button("Option 2");
option1.addActionListener(this); option2.addActionListener(this);
east.add(option1); east.add(option2);
add(east, BorderLayout.EAST);

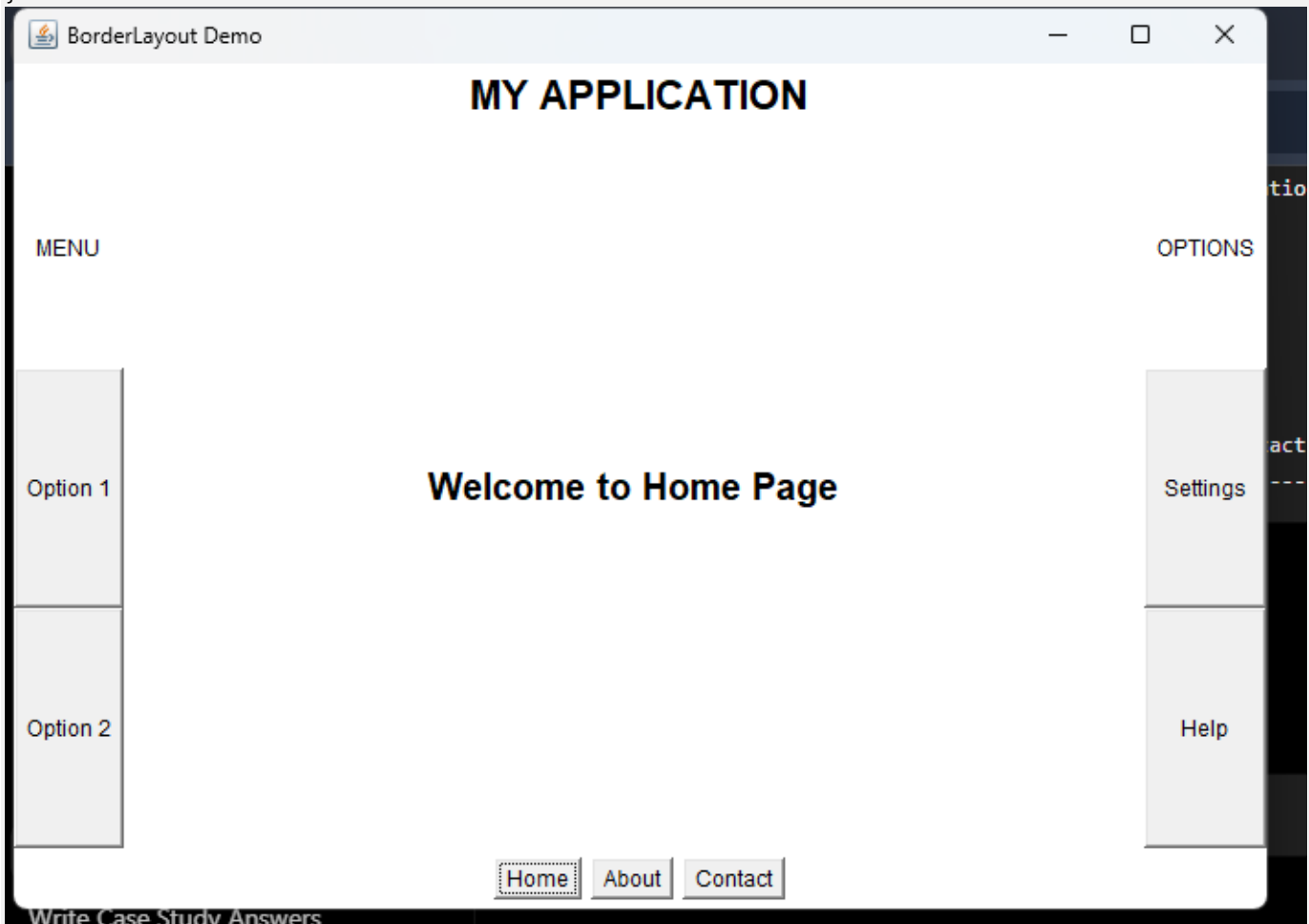
Panel west=new Panel(new GridLayout(3,1));
Button menu1=new Button("Menu 1");
Button menu2=new Button("Menu 2");
menu1.addActionListener(this); menu2.addActionListener(this);
west.add(menu1); west.add(menu2);
add(west, BorderLayout.WEST);

center.setFont(new Font("Arial",Font.BOLD,20));
add(center, BorderLayout.CENTER);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    center.setText("Selected: "+e.getActionCommand());
}

public static void main(String[] args) { new BorderDemo(); }
}

```



Result: The application demonstrates NORTH, SOUTH, EAST, WEST and CENTER regions with event-driven updates.

Question 8 – CardLayout with Event Handling

Aim: Create three cards—Student Information, Academic Information and Confirmation/Report—and navigate using Next, Previous and Submit.

```
import java.awt.*;
import java.awt.event.*;

public class CardDemo extends Frame implements ActionListener {
    CardLayout card = new CardLayout();
    Panel cards = new Panel(card);
    TextField name, reg, dept, year;
    int current = 1;

    CardDemo() {
        setTitle("Student Multi Page Form");
        setSize(500,300);

        Panel c1=new Panel(new FlowLayout());
        c1.add(new Label("Student Name:"));
        name=new TextField(25); c1.add(name);
        c1.add(new Label("Register No:"));
        reg=new TextField(15); c1.add(reg);

        Panel c2=new Panel(new FlowLayout());
        c2.add(new Label("Department:"));
        dept=new TextField(20); c2.add(dept);
        c2.add(new Label("Year:"));
        year=new TextField(10); c2.add(year);

        Panel c3=new Panel(new FlowLayout());
        c3.add(new Label("Confirmation / Report"));
        c3.add(new Label("Click Submit to confirm details.));

        cards.add(c1,"one"); cards.add(c2,"two"); cards.add(c3,"three");

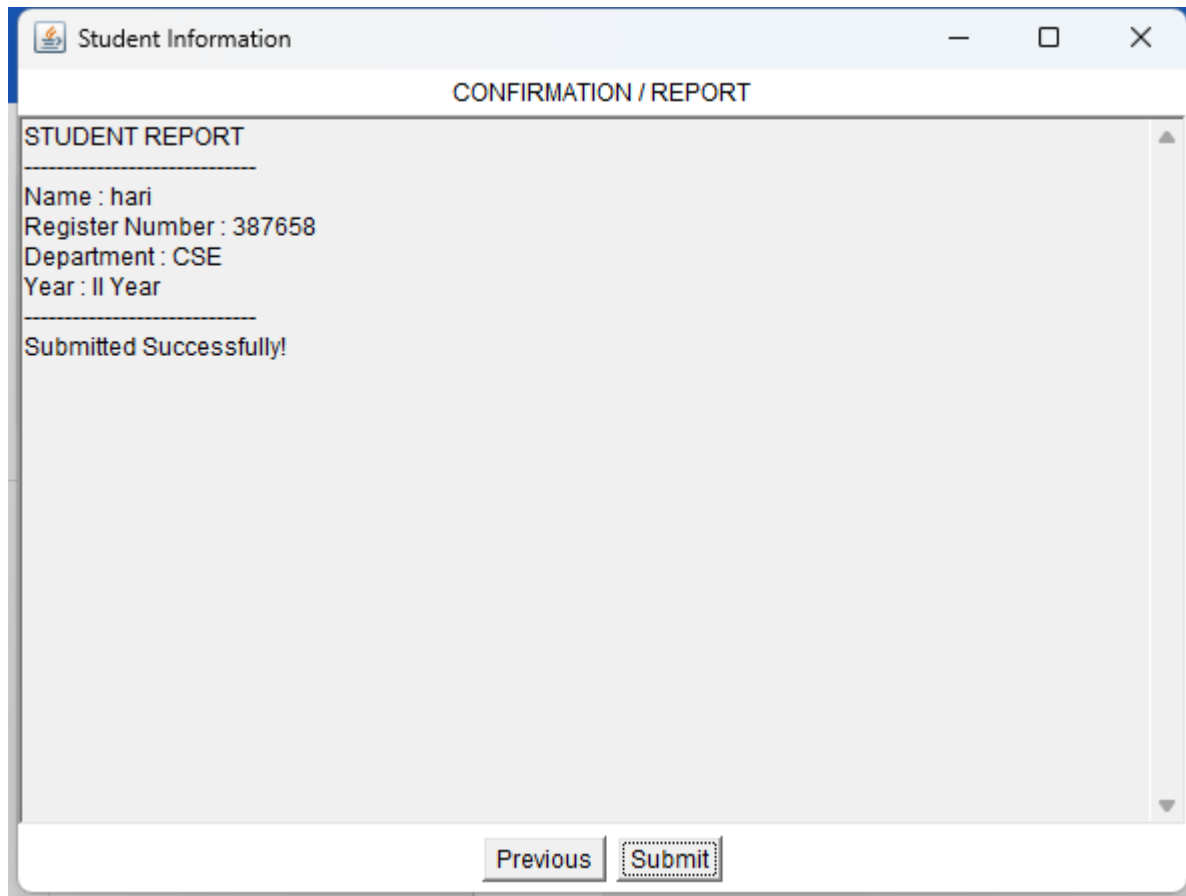
        Panel buttons=new Panel();
        Button prev=new Button("Previous");
        Button next=new Button("Next");
        Button submit=new Button("Submit");
        prev.addActionListener(this); next.addActionListener(this);
        submit.addActionListener(this);
        buttons.add(prev); buttons.add(next); buttons.add(submit);

        add(cards, BorderLayout.CENTER);
        add(buttons, BorderLayout.SOUTH);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        String s=e.getActionCommand();
        if(s.equals("Next") && current<3) { card.next(cards); current++; }
        else if(s.equals("Previous") && current>1) { card.previous(cards); current--; }
        else if(s.equals("Submit"))
            System.out.println("Report submitted: "+name.getText()+" / "+reg.getText());
    }
}
```

```
public static void main(String[] args) { new CardDemo(); }  
}
```

EXECUTION OUTPUT / SCREEN



Result: CardLayout switches between three screens in the same window and the navigation buttons work through ActionListener.

Lab Practice Evaluation Checklist

- Correct implementation of required Java concepts.
- Proper use of AWT components.
- Correct event handling.
- Appropriate layout managers.
- User-friendly interface.
- Successful compilation and execution.
- Ability to explain the program and output during the lab examination.

Note About Output Screens

The output panels above are execution-style representations of the expected Java AWT/Applet windows. They are intentionally formatted to resemble a simple IDE/runtime result rather than an AI-generated poster. For an actual lab submission, replace these panels with screenshots captured from your own successful execution if your faculty specifically requires real runtime screenshots.